



Trabajo práctico N°1

“Modelos de proceso”

Integrantes:

- FRANCISCO AGUSTIN BISSO - 166742
- RENZO MARTIN BISSO - 186104
- RODRIGO PALOMEQUE - 200967
- FAUSTINO GRASSO - 208183
- BAUTISTA ZUMPARO - 210664
- ANA PAULA ROMERO - 195388

11258/11058 - Sistemas de información II

ÍNDICE:

ÍNDICE:	1
INTRODUCCIÓN	2
AUTOR: Sommerville	3
Marco Teórico:	4
Actividades Fundamentales:	4
Modelos de Proceso:	4
Modelo en Cascada:	5
Desarrollo evolutivo:	6
Ingeniería de software basada en componentes:	8
Proceso unificado de Rational:	10
Ingeniería de software asistida por computadoras:	11
AUTOR: Pressman	12
OTROS AUTORES	20
Concepto del Modelo en Espiral	21
Fases de cada ciclo (Espirales)	21
Características Principales	21
Fase Dos: Actividad con Inteligencia Artificial	21
CONCLUSIONES	22
BIBLIOGRAFÍA Y REFERENCIAS	23

INTRODUCCIÓN:

El siguiente escrito pretende desarrollar las perspectivas de distintos autores sobre los procesos de desarrollo de software, incluyendo distintos modelos utilizados en dichos procesos, como fue consignado por los profesores de la materia. La investigación tiene como objetivo plantear las perspectivas de cada autor para que luego los alumnos encargados de este trabajo realicen sus propias conclusiones luego de haber realizado una comparación de las visiones de los distintos autores con la ayuda de una herramienta de Inteligencia Artificial Generativa.

AUTOR: Sommerville:

Marco Teórico:

La ingeniería de software se define como la disciplina que estudia los métodos, herramientas y técnicas para desarrollar software de alta calidad de manera sistemática y controlada.

Según Sommerville, *“un proceso de software es un conjunto estructurado de actividades necesarias para desarrollar un sistema de software. Estas actividades no son rígidas, sino que deben adaptarse al contexto del proyecto.”*

Actividades Fundamentales:

Sommerville define cuatro actividades esenciales presentes en todos los procesos de software:

1- **Especificación:** se definen los requisitos funcionales y no funcionales del sistema. Se estudia si el sistema que se tiene en mente en un principio es realmente necesario para el cliente y, en caso de que fuese así, se obtienen y analizan los requerimientos, se plantean en un documento que luego es verificado en busca de errores para su formalización.

2- **Diseño e Implementación:** se define la arquitectura del sistema y los subsistemas que lo compondrán, se diseña la interfaz que permita a los futuros usuarios utilizarlo sin necesitar conocimientos previos del sistema, y luego se asignan servicios a los distintos componentes (subsistemas) con sus respectivos algoritmos y estructuras de datos a manejar.

3- **Validación:** se realizan las pruebas del sistema, probando cada componente por separado y luego todos en conjunto en busca de errores. Luego se prueba ya con futuros usuarios, en lugar de con datos proporcionados por los desarrolladores, para comprobar que no haya más errores. Esta actividad suele ir a la par con el diseño.

4- **Evolución:** mantenimiento y adaptación a los cambios de requisitos del usuario del software tras su entrega. Debido a la gran flexibilidad de los sistemas de software, estos se pueden actualizar continuamente e incluso llegar a implementar subsistemas nuevos con el paso del tiempo.

Estas actividades constituyen el núcleo de cualquier proceso de desarrollo de software.

Modelos de Proceso:

Los modelos de proceso organizan las actividades del desarrollo de software. Entre los principales modelos se encuentran:

- **Modelo en cascada:** secuencial donde cada fase se completa antes de pasar a la siguiente.

- **Desarrollo evolutivo:** desarrollo en partes funcionales que se entregan progresivamente.

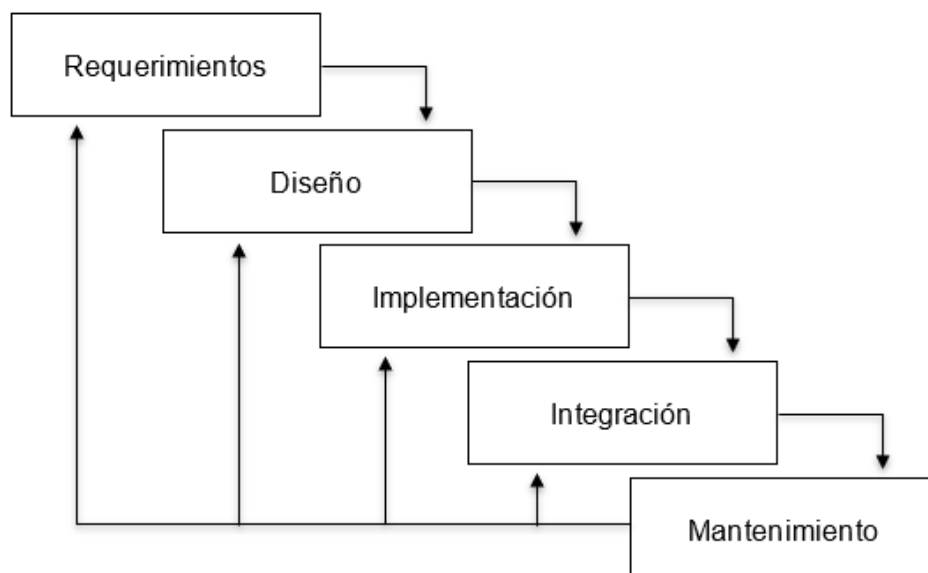
- **Ingeniería de software basada en componentes:** desarrollo basado en ciclos repetitivos de mejora.

Cada modelo tiene ventajas y limitaciones, y su elección depende del contexto del proyecto.

Modelo en Cascada:

Se caracteriza por ser un proceso secuencial en donde cada fase depende la anterior. Entre sus principales fases se destacan:

- 1- **Requerimientos (Análisis y definición de requerimientos):** Se obtienen los objetivos del sistema a partir de las consultas con los usuarios. Luego, se detalla y sirve como una especificación del sistema.
- 2- **Diseño (Del sistema y del Software):** Divide los requerimientos en hardware o software. Describe con abstracción el sistema y sus relaciones.
- 3- **Implementación y prueba (Unidades):** Se desarrolla el software como un conjunto o dividiendo lo en unidades. Las pruebas de unidades son para determinar si cumple o no con su requerimiento.
- 4- **Integración (Del sistema):** Se integran todas las unidades individuales en un mismo sistema y se prueba en su totalidad para asegurar que cumpla con todos los requerimientos. Luego de haber pasado las pruebas se entrega al usuario.
- 5- **Mantenimiento:** Fase más larga del ciclo de vida. El sistema se instala y se pone en funcionamiento. El mantenimiento implica la corrección de errores que no hayan podido solucionar anteriormente o no hayan sido descubiertos, mejorar la implementación de unidades del sistema y posteriormente satisfacer los próximos requerimientos del usuario.



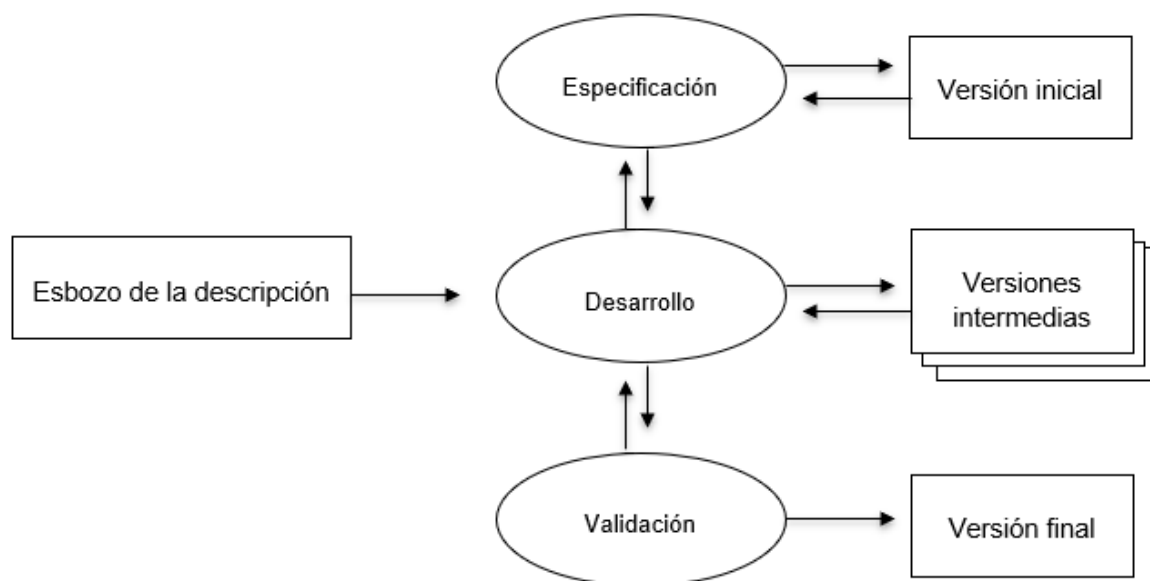
En la teoría la siguiente fase no debe empezar hasta que la anterior haya finalizado, en la práctica, estas etapas se suelen superponer proporcionando información las unas a las otras. El proceso de software no es un modelo lineal simple, sino que implica una serie de iteraciones de las actividades de desarrollo.

Las ventajas de este tipo de modelos son que la documentación se produce en cada fase y que está alineada con otros modelos del proceso de ingeniería. Pero su principal problema es la inflexibilidad al dividir el proceso en etapas.

Debido a eso, este modelo sólo se debe utilizar cuando se comprenden en su totalidad los requerimientos del usuario y sea improbable que cambien en gran medida en el futuro.

Desarrollo evolutivo:

Se basa en la idea de desarrollar un sistema inicial y someterlo al análisis y comentarios de los usuarios para luego ir refinándolo a través de diferentes versiones hasta finalizar con un sistema adecuado. La especificación, el desarrollo y la validación se entrelazan, con una retroalimentación rápida entre estas.



Existen dos principales tipos de este modelo:

- **Desarrollo exploratorio:** en donde se trabaja con el cliente para conocer sus necesidades y entregarle un sistema final. El desarrollo comienza con las partes del sistema que mejor se comprendan. El sistema va evolucionando continuamente agregando nuevos atributos que haya propuesto el usuario.
- **Prototipos desechables:** en donde el objetivo es comprender las necesidades del usuario y entonces desarrollar una definición mejorada de los requerimientos para el sistema. El prototipo está centrado en los requerimientos del usuario que no se comprende en su totalidad.

Este enfoque suele ser más efectivo que el modelo en cascada, ya que satisface las necesidades inmediatas del usuario. Su principal ventaja es la especificación creciente, cuando se consiga un mejor entendimiento de las necesidades del usuario se pueden reflejar en el sistema.

Sin embargo, desde la perspectiva de la ingeniería y de gestión, este modelo tiene dos principales problemas:

- **El proceso no es visible:** los desarrolladores deben de hacer entregas constantes para medir el progreso.
- **A veces los sistemas tienen una estructura deficiente:** los continuos cambios tienden a romper la estructura interna del software, por lo tanto, incorporar cambios se convierte en una tarea cada vez más difícil y costosa.

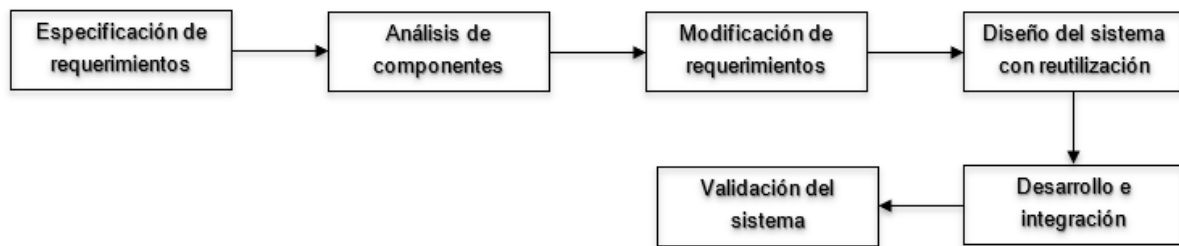
Este modelo está diseñado para sistemas pequeños o de mediano tamaño (hasta 500.000 líneas de código). Los problemas de este modelo se vuelven incontrolables en sistemas grandes y complejos con un período largo de vida, en donde diferentes equipos de producción desarrollan distintas partes del sistema.

Para sistemas grandes, se recomienda entrelazar el modelo en cascada con el desarrollo evolutivo. Esto implica desarrollar prototipos desechables para resolver incertidumbres en la especificación, en cambio las partes bien comprendidas el sistema se pueden desarrollar con el modelo en cascada.

Ingeniería de software basada en componentes:

Es común que, en los diseños de software, se reutilicen componentes de software ya realizados anteriormente. Por lo general, se busca y se modifica según sea necesario para agregarlo al sistema que se esté desarrollando en ese momento.

Esta reutilización es independiente del proceso de desarrollo que se utilice, sin embargo, en los últimos años, ha surgido un nuevo enfoque llamado Ingeniería de Software Basada en Componentes (CBSE) que se enfoca en la reutilización.



Este enfoque se compone de una gran base de componentes de software reutilizables y de algunos marcos de trabajo de integración para éstos. Estos componentes pueden ser también sistemas por sí mismos que se pueden utilizar para una funcionalidad específica.

Aunque existen muchas similitudes con respecto a las etapas de especificación y validación con otros modelos de procesos, las etapas intermedias son diferentes:

- 1- **Análisis de componentes:** Una vez obtenida la especificación de requerimientos, se buscan los componentes para implementar esta especificación.
- 2- **Modificación de requerimientos:** FALTA DEFINIR
- 3- **Diseño del sistema con reutilización:** Se diseña el sistema teniendo en cuenta los componentes reutilizados y se elabora o reutiliza un marco de trabajo para el mismo.
- 4- **Desarrollo e integración:** Para crear el sistema, todo el software que no se pudo adquirir se desarrolla, y los componentes y los sistemas se integran. En este modelo la integración de sistemas es una parte del proceso de desarrollo.

La ventaja obvia de este modelo es la reducción de cantidad de software a desarrollarse y por ende la reducción de costos y riesgos. Por lo general, también permite una entrega más rápida del software.

Iteración de procesos:

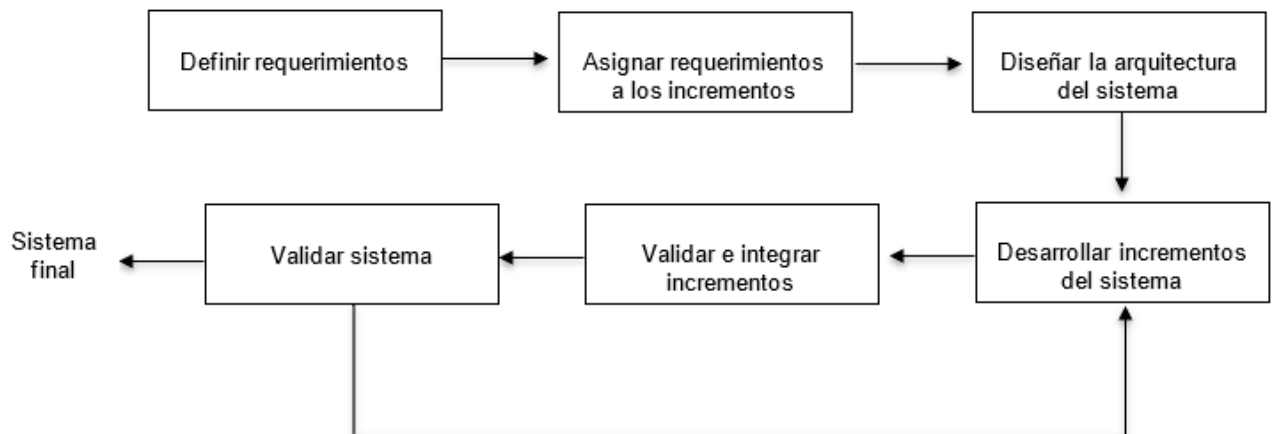
En los proyectos de software, los cambios son inevitables debido a nuevas tecnologías, presiones del negocio y la evolución de los requisitos. Por ello, el desarrollo no es un proceso lineal, sino iterativo, donde ciertas actividades se repiten conforme el sistema se ajusta.

Existen dos modelos principales que apoyan esta iteración:

- 1- **Entrega incremental:** el sistema se construye y se entrega en partes funcionales llamados incrementos.
- 2- **Desarrollo en espiral:** en donde el sistema se desarrolla en ciclos sucesivos que refinan progresivamente el software.

Entrega incremental:

Este modelo divide el desarrollo en incrementos que entregan una funcionalidad parcial pero útil. Los requisitos más importantes se implementan primero, y cada incremento añade nuevas capacidades.



Entre las principales ventajas se encuentran la entrega temprana al usuario. Retroalimentación continua que mejora los siguientes incrementos. Menor riesgo de fallo total del proyecto. Los servicios de más alta prioridad se entregan primero.

Algunas desventajas son la dificultad para manejar sistemas grandes o complejos. Problemas para identificar y gestionar recursos compartidos. Puede ser complicado ajustar requisitos a incrementos adecuados. También se ha desarrollado una variante llamada programación extrema, que enfatiza entregas frecuentes de incrementos de funcionalidad pequeños y participación del usuario.

Desarrollo en espiral:

Fue propuesto por Boehm, este modelo organiza el desarrollo en ciclos (espirales), donde cada ciclo incluye planificación, análisis de riesgos, desarrollo y evaluación.

Cada ciclo de la espiral se divide en cuatro secciones:

1. **Definición de objetivos:** en donde se establecen metas y alternativas
2. **Evaluación y reducción de riesgos:** se analizan posibles problemas y se buscan soluciones.
3. **Desarrollo y validación:** se construye y se prueba el sistema.
4. **Planificación:** se define si se continúa y cómo se avanza al siguiente ciclo.

La principal diferencia entre este modelo y los demás es la consideración del riesgo como parte del modelo. Un ciclo de la espiral empieza con la elaboración de objetivos, luego se encuentran formas alternativas de alcanzar estos objetivos y las posibles restricciones. Una vez evaluado los riesgos, se lleva a cabo el desarrollo, para continuar con una actividad de planificación para la siguiente parte del proceso.



Proceso unificado de Rational:

El Proceso Unificado de Rational (RUP) es un modelo moderno de desarrollo de software basado en el Lenguaje Unificado de Modelado (UML), que combina elementos de procesos genéricos y prácticas específicas de ingeniería de software.

RUP se desarrolla desde tres perspectivas:

- **Dinámica:** muestra las fases del modelo a lo largo del tiempo.
- **Estática:** presenta las actividades del proceso.
- **Práctica:** incluye buenas prácticas del desarrollo.

El proceso se organiza en cuatro fases principales:

1- **Inicio:** Define el caso de negocio, identifica actores externos y evalúa la viabilidad del proyecto.

2- **Elaboración:** Analiza el dominio del problema, establece la arquitectura del sistema, desarrolla un modelo de requisitos en UML y evalúa riesgos.

3- **Construcción:** Incluye el desarrollo diseño, programación y pruebas del sistema hasta obtener un producto funcional.

4- **Transición:** Entrega el sistema a los usuarios finales, asegurando su correcto funcionamiento en el entorno real.

La característica principal del RUP es que es un proceso iterativo e incremental, donde los resultados se desarrollan en ciclos. Integra las fases mediante un enfoque continuo no estrictamente lineal. Se apoya en flujos de trabajo, como modelado, implementación y pruebas, que pueden estar presentes en varias fases.

Ingeniería de software asistida por computadoras:

La Ingeniería de Software Asistida por Computadora (CASE) se refiere a herramientas de software que apoyan distintas fases del desarrollo de software, como el análisis de requisitos, diseño, construcción y pruebas. Estas herramientas incluyen editores de diseño, compiladores, depuradores, sistemas de construcción, entre otros.

CASE ayuda a automatizar el proceso de desarrollo y a gestionar la información del software en construcción. Dentro de las tareas que puede automatizar se destacan:

- Modelado gráfico del sistema.
- Diseño de estructuras de datos.
- Generación de interfaces de usuarios.
- Producción de programas a partir de especificaciones.
- Conversión automática entre lenguajes.

Se esperaba que CASE redujera en gran medida los costos, pero los resultados han sido menores a lo previsto (menos del 40%), esto debido a la dificultad para automatizar el diseño y la necesidad de interacción humana en equipos de desarrollo.

Clasificación de CASE:

Las herramientas CASE pueden clasificarse según tres perspectivas:

- **Funcional:** según su función específica en el proceso.
- **Proceso:** según las actividades del desarrollo que apoyan.
- **Integración:** según cómo se organizan e integran para apoyar múltiples actividades.

Otra clasificación distingue entre:

- **Herramientas:** apoyan tareas específicas (por ejemplo, edición o compilación).
- **Bancos de trabajo (workbenches):** integran varias herramientas para una fase del proceso (como diseño o especificación).

También existen:

- **Herramientas de propósito general:** apoyan diversas actividades a lo largo del proceso, incluyendo modelado, control de versiones y gestión de datos.
- **Herramientas especializadas o de propósito específico:** enfocadas en tareas concretas.

Finalmente, es importante señalar que las herramientas pueden venderse individualmente o como conjuntos integrados. Los bancos de trabajo suelen centrarse en áreas como programación o diseño. No siempre es fácil clasificar una herramienta, pero la clasificación ayuda a comprender su alcance dentro del proceso de software.

AUTOR: Pressman

Modelo del proceso:

Elaboración de software como proceso reiterativo de aprendizaje social, la reunión de conocimiento recabado, depurado y organizado a medida que se realiza el proceso.

Proceso de software como una estructura para las actividades, acciones y tareas que se requieren a fin de construir software de alta calidad.

La ingeniería de software es lo que se hace sobre el software, incluye métodos técnicos y herramientas automatizadas.

IMPORTANCIA: da estabilidad, control y organización a una actividad que puede volverse caótica si se descontrola.

Estructura general para la Ingeniería de Software:

Define 5 actividades.

- Comunicación
- Planeación
- Modelado
- Construcción
- Despliegue

Lo que plantea es que, en cada etapa, haya un conjunto de actividades sombrilla de seguimiento y control del proyecto, administración de riesgos, seguimiento de la calidad, administración de configuración, revisiones técnicas, entre otras. Cada una deberá escoger el conjunto de tareas que se adapte mejor a las necesidades del proyecto y características del equipo.

Existe un FLUJO DE PROCESO, el cual describe la manera en la que están organizadas las actividades estructurales y las acciones y tareas que ocurren dentro de cada una con respecto de la secuencia y el tiempo.

Actividad estructural e identificación de tareas:

Para poder desarrollar cada una de las etapas estructurales, se necesitará información para saber qué acciones son apropiadas llevar según el problema a resolver, antes de ejecutar cualquiera de las mismas.

Si hablamos de un proyecto pequeño y sencillo, bastará con una conversación, donde exista el contacto, se analice los requerimientos, se tomen notas, hacer una nota por escrito de una formulación breve de los requerimientos y enviarlos para su aprobación.

Si es considerablemente más complejo, ya se habla de 6 acciones distintas: (las cuales tendrán sus propias tareas y entregas)

1. Concepción
2. Indagación
3. Elaboración

4. Negociación
5. Especificación
6. Validación

Patrones del proceso:

Es importante contar con **patrones del proceso**, los cuales permiten describir un problema relacionado con el proceso que se encuentra durante el trabajo, lo identifica, identifica el ambiente en el que surgió y propone soluciones para el mismo.

Puede ser utilizado sobre un modelo completo del proceso o asociado a una actividad estructural específica.

Comienzan en un nivel alto de abstracción permitiendo desarrollar una descripción jerárquica del proceso.

Formato propuesto por Ampler:

- **Nombre del patrón**
- **Fuerzas:** ambiente en el que se encuentra, aspectos que lo hacen visible y afectan su solución.
- **Tipo:**
 - Patrón de etapa: incorpora múltiples patrones de la tarea.
 - Patrón de tarea: define un problema asociado con una acción o tarea de trabajo.
 - Patrón de fase: define secuencia de las actividades estructurales que ocurren dentro del proceso. Aún cuando el flujo es iterativo.
- **Contexto inicial:** describir condiciones en la que se aplica el patrón.
- **Problema**
- **Solución**
- **Contexto resultante**
- **Patrones relacionados.**

Evaluación y mejora

A pesar de que siempre se toman medidas para poder crear cada etapa estructural de la manera óptima posible, siempre deben hacerse evaluaciones y mejoras, para esto se proponen diferentes enfoques como:

- Método de evaluación del estándar CMMI para el proceso de mejora: inicio, diagnóstico, establecimiento, actuación y aprendizaje.
- Evaluación basada en CMM para la mejora del proceso interno: diagnóstico de madurez relativa de la organización.
- SPICE: define conjunto de requerimientos para la evaluación.
- ISO9001:2000 para sw. Estándar genérico.

Modelos de proceso prescriptivo

Propuestos para poner orden en el caos del desarrollo de sw. Prescriben un conjunto de elementos del proyecto. Pueden incluir las actividades estructurales pero cada una pone distinto énfasis en ellas y define en forma diferente el flujo de proceso que invoca cada actividad.

1. Cascada: Mayormente encontrada en ocasiones donde deben hacerse adaptaciones o mejoras bien definidas a un sistema ya existente.
 - a. Una variante es el *modelo en V*, relación entre las acciones para asegurar la calidad.
 - b. Problemas:
 - i. Los proyectos reales rara vez siguen un flujo secuencial
 - ii. Es difícil para el cliente enunciar todos los requerimientos, hay dificultad de interacción con el usuario, para tener su feedback.
 - iii. Cliente solo participa en el principio y cuando el programa ya está casi listo.
 - iv. Puede encontrarse con estados de bloqueo, en el que ciertos miembros del equipo deben esperar a otros a fin de terminar tareas independientes.
2. Proceso incremental: combina elementos de los flujos lineal, pero de manera escalonada.
 - a. Cada secuencia produce avances o incrementos susceptibles de entregarse.
 - b. Es frecuente que el primer incremento sea el producto fundamental. Se consulta con el cliente y en base a esa devolución, se desarrolla un plan para el incremento que sigue.
 - c. Es útil particularmente cuando no se dispone de personal para la implementación completa del proyecto en el plazo establecido.
 - d. Los incrementos socorran los riesgos técnicos.
3. Proceso evolutivo: Se basa en la entrega de una versión limitada a fin de aliviar la presión de la competencia o el negocio, con detalles o extensiones del sistema que aún están por definirse.
 - a. Son iterativos.
 - b. HACER PROTOTIPOS:
 - i. Técnica cuando los requerimientos no están claros. Cuenta una versión inicial simple para entender qué se necesita construir.
 - ii. 1° comunicación con los usuarios para definir objetivos generales, 2° diseño rápido enfocado en cómo el usuario va a ver. 3° se muestra a usuarios (feedback), 4° a partir de eso se evoluciona el prototipo.
 - iii. Puede usarse para entender el sistema y luego descartarse o evolucionar hasta el proceso final.
 - iv. Ventajas: visualización para clientes y avance rápido para desarrolladores
 - v. Desventajas: los usuarios pueden creer que el prototipo ya es el producto final y por otro lado, los desarrolladores pueden tomar decisiones rápidas que luego no son adecuadas.
 - c. MODELO ESPIRAL:
 - i. Combina iteración del prototipo con la estructura del modelo en cascada. Enfoque evolutivo, se desarrolla en ciclos sucesivos, generando versiones cada vez más completas.
 - ii. Está orientado al manejo de riesgos, en cada ciclo se analizan, reducen y controlan antes de avanzar.
 - iii. Cada iteración incluye planificación, desarrollo, evaluación y ajustes según la retroalimentación del cliente.

- iv. La diferencia importante es que el modelo espiral se puede aplicar durante toda la vida del sw.
 - v. Ventajas: adaptación mejor a cambios, manejo de riesgos y construcción de sistemas complejos de forma progresiva.
 - vi. Desventajas: requiere mucha experiencia en análisis de riesgos y puede ser difícil de aplicar o justificar en proyectos con clientes que esperan procesos más rígidos.
4. Modelos concurrentes: Permite al equipo de desarrollo presentar elementos iterativos y concurrentes de cualquiera de los momentos de proceso descritos. Permiten que ocurran al mismo tiempo, en lugar de seguir una secuencia rígida.
- a. Cada actividad no avanza de forma lineal, sino que puede estar en distintos estados. Estas actividades cambian de estado según eventos, como modificaciones en los requerimientos.
 - b. Por ejemplo, mientras una parte del sistema está en desarrollo, otra puede estar en revisión y otra esperando cambios. Si el cliente pidiera una modificación, una actividad determinada puede volver a un estado anterior (Cambios en espera).
 - c. Este modelo representa el proceso como una **red de actividades simultáneas**, donde todo está conectado y puede evolucionar en paralelo.
 - d. Ventaja principal: refleja mejor cómo se trabaja en proyectos de sw, mayor flexibilidad, mejor adaptación a cambios, miembros del equipo no quedan esperando que otros terminen para poder avanzar.

Conclusiones sobre los procesos evolutivos:

Los procesos evolutivos surgen para adaptarse a un contexto donde hay cambios constantes, plazos ajustados y fuerte necesidad de satisfacer al cliente, especialmente cuando los tiempos y la presión del mercado son críticos.

Sin embargo, aún continúan con algunas debilidades:

- Es difícil planificar el proyecto porque no se puede predecir la cantidad de iteraciones necesarias.
- No hay un ritmo claro de evolución.
- Se prioriza la flexibilidad y la rapidez por sobre la calidad.

Aún así, el desafío principal es encontrar un equilibrio entre velocidad, flexibilidad, calidad y satisfacción del cliente.

Modelos de proceso especializado:

Utilizados cuando los modelos tradicionales no son suficientes, cuando se elige un enfoque especializado de la ingeniería de sw.

Incluye:

1. Desarrollo basado en componentes:
 - a. Reutiliza componentes de sw ya existentes, en lugar de construir todo desde cero, el sistema se arma integrando piezas reutilizables.
 - b. Ventajas: reduce tiempo, costo y mejora la confiabilidad (porque los componentes ya fueron probados).
2. Métodos formales:

- a. Utilizan matemática y lógica formal para especificar, diseñar y verificar el software. Permiten demostrar que el sistema es correcto antes de implementarlo.
 - b. Se usan en sistemas críticos (ej: aeronáutica, medicina).
 - c. Desventaja: son complejos y requieren mucha especialización.
3. Orientado a objetos:
- a. El DSOA trata las preocupaciones globales (seguridad, memoria, transacciones, etc.) que afectan a todo el sistema.
 - b. Estas se encapsulan como aspectos, separados de los componentes tradicionales.
 - c. Evita que esas funcionalidades estén dispersas y duplicadas en el código.
 - d. Mejora la organización, mantenimiento y evolución del software.
 - e. Los aspectos se integran con los componentes y pueden afectar varias partes del sistema.
 - f. Suele combinar ideas de modelos evolutivos y concurrentes (se desarrollan en paralelo).

Estos modelos no reemplazan a los tradicionales, sino que se usan cuando el proyecto lo requiere, aportando:

- Reutilización (componentes)
- Rigor matemático (métodos formales)
- Organización moderna y adaptable (proceso unificado)

El proceso unificado:

Es un intento por obtener los mejores rasgos y características de los modelos tradicionales del proceso de software, implementando los mejores principios del desarrollo ágil, los métodos individuales de análisis y diseño orientado a objetos. Hace foco en la arquitectura de sw, a través de un proceso iterativo e incremental.

- a. El resultado fue un UML, lenguaje de modelado unificado. El proceso unificado es la estructura del proceso que guía a los equipos a llevar a cabo la práctica de ingeniería de software orientado a objetos.
- b. Fases: como otros modelos de procesos también, cada fase estructural se relaciona con el PU
 - a. **Concepción:** comunicación con el cliente, recolección de requisitos y planeación. Los requisitos se describen mediante un conjunto de casos de uso preliminares.
 - b. **Elaboración:** incluye comunicación y modelado del modelo general del proceso. Incluyen: modelos del caso de uso, de requerimientos, del diseño, implementación y despliegue. Acerca una primera arquitectura ejecutable pero no proporciona todas las características y funciones que se requieren.
 - c. **Construcción:** Adquiere los componentes del sw que harán que cada caso de uso sea operativo para los usuarios finales. Comienza con el desarrollo de código fuente para las funcionalidades necesarias. Se realizan pruebas y ensambles de componentes.

- d. **Transición:** Se realizan detalles de la construcción y se les realizan entregas a los usuarios para su feedback. Se genera información de apoyo necesaria para la utilización del sistema. Ya se obtiene un producto utilizable.
- e. **Producción:** Forma parte de la fase de despliegue, donde se evalúa el uso del sistema, su ambiente, se realizan mejoras y reciben solicitudes de cambios.
- c. Este modelo, no se lleva a cabo de manera secuencial, sino escalonada. Se organiza en flujos de trabajo distribuidos en todas las fases. Estos flujos de trabajo son conjunto de tareas requeridas y productos resultantes.
- d. El proceso se adapta según el proyecto. Los equipos adaptan actividades y tareas según necesidades específicas.

Modelos del proceso personal y del equipo:

Este modelo mantiene la premisa de cubrir las necesidades del equipo que lleva a cabo el proyecto y la organización, por ende. Pretender crear un proceso personal de sw y/o un proceso del equipo de sw. Para eso se requerirá de capacitación y coordinación.

- a. PPS (Proceso personal del sw): Resume en 4 fases, lo que todo desarrollador debe cumplir para lograr un proceso de sw eficiente y de calidad. Actualmente no es ampliamente utilizado debido al costo de la capacitación requerida.
 - a. **Planeación:** Aísla requerimientos, desarrolla estimaciones del tamaño como de los recursos que serán necesarios.
 - b. **Diseño de alto nivel:** especificaciones y diseño de y para cada componente. Incluye elaboración de prototipos.
 - c. **Desarrollo:** se mejora y revisa el diseño del componente. Se genera el código fuente.
 - d. **Post Mórtem:** A través de ediciones se analiza la eficiencia del proceso.
- b. PES (Proceso del equipo de sw): Es una extensión del PPS.
 - a. Forma equipos autodirigidos que planeen y den seguimiento del trabajo.
 - b. Muestra al sector gerencial cómo dirigir, motivar y ayudar a los equipos.
 - c. Acelera optimización del proceso.
 - d. Proporciona guías de mejora
 - e. Facilita la enseñanza universitaria de las respectivas aptitudes necesarias como equipos.
 - f. Se logra una integración, compromiso y participación óptima de los miembros del equipo.
 - g. Define: un inicio, diseño, implementación, integración y pruebas y post mórtem.
 - h. Utiliza scripts, formatos y estándares como guía.

Tecnología del proceso

Para lograr una adaptación de uno o más modelos se desarrollaron las herramientas de tecnología del proceso, con el fin de analizar el proceso actual, organizar, controlar y vigilar el avance.

Favorece la creación de un modelo automatizado y adaptado a cada organización y caso particular.

Existen también herramientas para asignar, vigilar e incluso controlar todas las actividades, acciones y tareas.

Producto y proceso

Se plantea que en ingeniería de sw se debe entender el *producto* y el *proceso* como una dualidad, ambos son igualmente importantes y se complementan. Un mal proceso afecta al producto final, pero centrarse demasiado en el proceso también puede ser perjudicial.

A lo largo del tiempo, la industria ha ido alternando su enfoque entre aspectos del producto (como tecnologías o lenguajes) y del proceso (métodos y modelos), generando confusión sin resolver realmente los problemas.

La idea clave es que no se puede comprender ni mejorar el software considerando sólo uno de los dos. Producto y proceso están interrelacionados, y equilibrarlos permite mejorar la calidad, la reutilización y el desarrollo en general. Además, el desarrollo de software también tiene un componente creativo, por lo que los profesionales encuentran valor tanto en el proceso como en el resultado final.

Conclusión

El proceso de software es una estructura que organiza el desarrollo a través de actividades como comunicación, planeación, modelado, construcción y despliegue, junto con tareas y productos asociados. Estas actividades pueden organizarse de distintas formas según el modelo de proceso, lo que define cómo se avanza en el desarrollo, ya sea de manera lineal, iterativa, evolutiva o en paralelo. Además, los patrones de proceso permiten resolver problemas comunes reutilizando soluciones ya conocidas.

Los modelos prescriptivos, como cascada y modelo en V, buscan orden y estructura mediante un flujo secuencial, siendo útiles cuando los requerimientos están bien definidos, aunque poco flexibles frente a cambios. En cambio, los modelos incrementales y evolutivos se adaptan mejor al contexto actual, ya que permiten desarrollar el software en partes o iteraciones, generando versiones cada vez más completas y facilitando la retroalimentación del usuario. Dentro de estos, el modelo espiral se destaca por incorporar el análisis de riesgos en cada ciclo, mientras que el prototipo ayuda a comprender mejor los requerimientos cuando no están claros.

El modelo concurrente refleja una forma más realista de trabajo, ya que permite que distintas actividades ocurran al mismo tiempo y en diferentes estados. Por otro lado, los modelos especializados abordan necesidades específicas, como la reutilización de componentes, el uso de métodos formales para asegurar la corrección del software, el desarrollo orientado a aspectos para manejar funcionalidades globales, y el Proceso Unificado, que combina iteración, enfoque en la arquitectura y casos de uso.

Los procesos evolutivos surgen como respuesta a la necesidad de adaptarse al cambio y a tiempos de entrega rápidos, aunque presentan desafíos como la dificultad de planificación y el equilibrio entre velocidad y calidad. En este contexto, es fundamental entender que producto y proceso no son opuestos, sino una dualidad: ambos se complementan y son necesarios para lograr software de calidad. Un buen desarrollo depende tanto de cómo se construye el software como del resultado final, y los profesionales encuentran valor tanto en el proceso creativo como en el producto obtenido.

OTROS AUTORES

A Spiral Model of Software Development and Enhancement (Barry Boehm)

Concepto del Modelo en Espiral

El modelo en espiral es un enfoque de desarrollo de software que combina elementos del modelo en cascada con la naturaleza iterativa de la construcción de prototipos. Su principal objetivo es gestionar y minimizar los riesgos del proyecto mediante evaluaciones continuas en cada ciclo.

Fases de cada ciclo (Espirales)

En cada iteración o "vuelta" de la espiral, el equipo de desarrollo atraviesa cuatro sectores o fases principales:

1. Planificación / Determinación de Objetivos: Se definen las metas del producto, las restricciones de gestión y las alternativas posibles.
2. Análisis de Riesgos: Se evalúan las alternativas y se identifican los puntos críticos que podrían hacer fracasar el proyecto. Aquí es donde se suelen crear prototipos para resolver incertidumbres técnicas.
3. Ingeniería / Desarrollo: Se genera el código y se realizan las pruebas de la versión actual del software.
4. Evaluación: El cliente inspecciona los resultados del ciclo actual y, si se aprueba, se planifica la siguiente fase de la espiral.

Características Principales

- Evolución de la Complejidad: Con cada ciclo, aumenta el tiempo de ejecución y el volumen de código fuente desarrollado.
- Crecimiento en Gestión: La planificación y la gestión de riesgos se vuelven más complejas a medida que el proyecto avanza.
- Flexibilidad: A diferencia del modelo "code and fix" (codificar y corregir), este permite una evolución estructurada y consciente de los peligros potenciales del desarrollo.

Comparación de Modelos con Inteligencia Artificial

La inteligencia artificial elegida fue Gemini de Google. Las comparaciones entre autores y modelos partieron desde los archivos de referencia dados de Sommerville y Pressman en el aula virtual.

A continuación, se detallan los prompts y respuestas obtenidas.

1. Separando por autores, ¿Cuál es la perspectiva de cada uno?

Perspectiva de Ian Sommerville: Define el proceso como un conjunto de actividades intelectuales y creativas que dependen del juicio humano. Su enfoque es técnico y divide el proceso en cuatro actividades fundamentales comunes a todos:

- Especificación de software: Definir la funcionalidad y restricciones.
- Diseño e implementación: Producir el software que cumple con la especificación.
- Validación: Asegurar que el sistema haga lo que el cliente desea.
- Evolución: Modificar el software para cubrir necesidades cambiantes.

Perspectiva de Roger Pressman: Presenta el proceso como un diálogo y un proceso de aprendizaje social. Su estructura general para la ingeniería de software define cinco actividades estructurales:

- Comunicación
- Planeación
- Modelado
- Construcción
- Despliegue

2. ¿Cuáles son los modelos y conceptos clave de cada autor?

Modelos destacados de Ian Sommerville:

- Modelo en Cascada: Fases separadas y secuenciales (requerimientos, diseño, implementación, etc.). Es útil cuando los requerimientos son estables.
- Desarrollo Evolutivo: Entrelaza la especificación, el desarrollo y la validación. Ideal para sistemas pequeños o de tamaño medio.
- Ingeniería Basada en Componentes: Se enfoca en integrar componentes reutilizables en lugar de desarrollar desde cero.
- Tecnología CASE: Herramientas para automatizar actividades rutinarias, aunque su eficacia está limitada por la diversidad de procesos y la creatividad necesaria.

Modelos y Conceptos Clave de Roger Pressman:

- Flujos de Proceso: Define que las actividades pueden ser lineales, iterativas, evolutivas o paralelas.
- Modelo en V: Una variante del modelo en cascada que vincula las acciones de verificación y validación con las fases tempranas de ingeniería.
- Desarrollo en Espiral: Un modelo impulsado por el riesgo que desarrolla el software en una serie de entregas evolutivas cada vez más completas.

- Modelos Especializados: Incluye el Desarrollo Orientado a Aspectos para preocupaciones globales (como seguridad) y Métodos Formales basados en matemáticas para sistemas críticos.
 - Procesos Personales y de Equipo (PSP/TSP): Enfatiza la medición individual, la planeación y la formación de equipos autodirigidos para mejorar la productividad y calidad.
3. Comparativa entre los modelos de proceso de estos dos autores.
- Modelos secuenciales. Ambos describen el modelo de cascada como el paradigma más antiguo, derivado de la ingeniería de sistemas.
 - **Sommerville**: Lo describe como fases separadas (requerimientos, diseño, implementación, integración y mantenimiento). Enfatiza que es inflexible y solo debe usarse cuando los requerimientos se comprenden bien.
 - **Pressman**: Introduce una variante importante llamada "Modelo en V". Este modelo ilustra como las acciones de verificación y validación (pruebas) se asocian directamente con las etapas iniciales de ingeniería.
 - Modelos Incrementales e Iterativos
 - Desarrollo Incremental:
 - **Sommerville**: Lo ve como una forma de entregar servicios por orden de prioridad, permitiendo al cliente usar el sistema tempranamente.
 - **Pressman**: Lo define como una serie de avances (incrementos) que entregan un "producto fundamental" (versión desnuda) en la primera etapa, al cual se le agregan funciones en etapas posteriores.
 - Modelo en Espiral:
 - **Sommerville**: Se divide en cuatro sectores: definición de objetivos, evaluación de riesgos, desarrollo/validación y planificación. El enfoque principal es la **consideración explícita del riesgo**.
 - **Pressman**: Lo describe como un "generador de modelos" impulsado por el riesgo, diseñado para sistemas de gran escala donde el software evoluciona en cada circuito de la espiral.

CONCLUSIONES

El análisis de los distintos modelos de proceso propuestos por Ian Sommerville y Roger Pressman permite comprender cómo la ingeniería de software ha intentado organizar y sistematizar el desarrollo de sistemas. Sin embargo, al contrastar estas propuestas con la visión de Frederick P. Brooks, surge una limitación fundamental: la imposibilidad de resolver la complejidad del software mediante un único enfoque o herramienta.

La intención de Sommerville y Pressman es clara: su propósito fue desarrollar herramientas y modelos útiles que acompañen y guíen a los desarrolladores y equipos en la elaboración de procesos de software. Para ello, realizaron un análisis exhaustivo de los principales modelos de proceso a lo largo del tiempo, identificando sus ventajas, desventajas y limitaciones.

A partir de esto, se evidencia una evolución tanto en las herramientas como en la filosofía adoptada en la ingeniería de software. Esto resulta de gran importancia, ya que permite reconocer que muchos modelos ampliamente utilizados en su momento no lograron cumplir completamente con las expectativas iniciales, dando lugar a un proceso continuo de mejora. En este contexto, se destaca la importancia de la interacción con el usuario, cuyo feedback resulta fundamental para reducir el impacto de factores como la dificultad para expresar requisitos, la omisión de necesidades o los cambios constantes durante el desarrollo.

Asimismo, se puede observar que esta evolución no ha sido lineal. A lo largo del tiempo han surgido modelos y herramientas considerados “soluciones definitivas”, pero que con el tiempo demostraron limitaciones. En este punto cobra relevancia el aporte de Frederick P. Brooks, quien en su ensayo “No hay balas de plata” reflexiona sobre los límites inherentes de la ingeniería de software.

Brooks propone analizar la verdadera complejidad del software, a la que denomina “esencia”, y la explica mediante la analogía del hombre lobo y la bala de plata. En este sentido, la bala de plata representa la búsqueda de una solución mágica capaz de resolver todos los problemas del desarrollo, mientras que el hombre lobo simboliza aquellos proyectos que, debido a su complejidad, se vuelven difíciles de controlar.

El autor distingue entre dificultades “accidentales” y “esenciales”. Las primeras corresponden a aspectos técnicos que pueden ser automatizados o mejorados mediante herramientas, como el uso de lenguajes de alto nivel, la generación de código o la detección de errores. En cambio, las dificultades esenciales están relacionadas con la comprensión del problema, la definición de requisitos y el diseño del sistema, constituyendo el verdadero desafío del desarrollo de software.

En este sentido, si bien los modelos propuestos por Sommerville y Pressman buscan organizar y controlar el proceso, no logran resolver completamente estas dificultades esenciales. Esto se evidencia en la continua aparición de nuevos modelos, lo que demuestra que ninguno resulta totalmente suficiente. Un ejemplo claro es el caso de las herramientas CASE, que intentaron automatizar el desarrollo, pero cuyo impacto fue menor al esperado. Esto se debe a que, como plantea Brooks, dichas herramientas solo abordan problemas accidentales, mientras que la mayor complejidad reside en la recolección y refinamiento de requisitos.

Finalmente, el software presenta características como complejidad, cambio constante, conformidad e invisibilidad, que dificultan su desarrollo y limitan el alcance de los modelos y herramientas existentes. Si bien estos resultan de gran utilidad para organizar y mejorar el proceso, no constituyen soluciones definitivas. En línea con Brooks, no existen “balas de plata”, sino avances graduales que permiten enfrentar, pero no eliminar, la complejidad inherente al software. Por lo tanto, es fundamental utilizar estas herramientas de manera consciente, comprendiendo sus límites y evitando la búsqueda de soluciones mágicas que prometan resolver completamente el problema.

BIBLIOGRAFÍA Y REFERENCIAS

- Sommerville, I. (s. f.). *Procesos*. Texto proporcionado por los profesores a través del aula virtual.
- Pressman, R. S. (s. f.). *Procesos*. Texto proporcionado por los profesores a través del aula virtual.
- Wikipedia contributors. (s. f.). *Desarrollo en espiral*. En *Wikipedia, la enciclopedia libre*. Recuperado el 28 de marzo de 2026, de https://es.wikipedia.org/wiki/Desarrollo_en_espiral